

这个自制的live2d模型的手会不会小了?

gpt-5.5-instant

我觉得**手确实有一点偏小了，尤其是右边举起来的那只手**，不过还没到特别违和的程度。

仔细看了一下这个 Live2D 模型：

- **画面右侧举起的手**：手掌稍微偏小，手指也比较短，看起来有点像小孩子的手。
- **画面左侧自然下垂的手**：也有些偏小，不过因为袖子比较宽大，所以不太明显。
- **手腕部分**：袖口比较宽，手掌又比较小，导致手从袖子里伸出来时显得有点迷你。

考虑到你的模型是偏可爱的二次元画风，手本来就不需要严格遵循真实人体比例。

我的建议是：

- 手掌整体放大 **10% ~ 15%**。
- 手指可以稍微加长一点，尤其是举起来的那只手。
- 不用改动手臂和袖子的大小，目前袖子的比例挺协调的。

另外有个比较实用的判断方法：

二次元角色的手掌长度（从手腕到中指指尖），通常可以参考角色从下巴到眉毛附近的距离。

你这个模型的手明显比这个参考尺寸小一些。

不过我还注意到一点：**相比手掌本身，手掌和袖口之间的大小反差可能才是让手显小的主要原因。**

如果是我来调整，我会先尝试把双手放大 **15%**，看看整体效果。

手掌形状和样子有没有错误?

gpt-5.5-instant

从这张图来看，**手掌的形状基本没有明显的结构性错误，但有几个地方可以优化。** 因为图片分辨率比较低，部分手指的细节看不太清楚。

1. 画面右侧举起来的手

这只手的问题相对明显一些：

- **手指偏细、偏短**：尤其是中指和无名指，长度差异不太明显。
- **手掌偏窄**：手掌与手指的比例有点不协调，感觉手指直接从手腕附近伸出来。
- **拇指的位置**：拇指与其他手指的分离程度看起来基本合理，但拇指根部可以画得更饱满一点。
- **手指张开的姿势**：整体能看出是在挥手，不过手指之间的间距和角度可以更自然一些。

2. 画面左侧垂下的手

这只手的问题主要是：

- 手掌略显狭长，掌心部分不够饱满。
- 手指之间的区分不够明显。
- 手腕到手掌的过渡稍微有点生硬。

不过，这只手的姿势比较自然，没有明显的骨骼结构错误。

3. 有没有画错手指?

从目前的图片来看，我没有发现明确的多指或缺指问题。

但由于手部像素比较少，没办法准确判断每根手指的连接方式。

4. 最值得修改的地方

我认为最需要改的是画面右侧举起的手。

它的问题并不只是小，而是**手掌部分不够明显，导致整只手看起来更像几根细手指从袖口伸出来。**

建议适当增加掌心的宽度和长度，再微调手指比例。

另外，我想确认一个细节：**你这只举起来的手，是想画成手心朝向观众的挥手姿势，还是手背朝向观众?**

这两种姿势的拇指位置和手指结构不一样，确认后可以更准确地判断有没有画错。

这种风格的live2d和nuerosama的区别是什么?

gpt-5.5-instant

你这个 Live2D 模型和 Neuro-sama 的 Live2D 模型，其实都属于**偏可爱的日系二次元画风**，但在角色比例、线条、上色和整体视觉表现上有一些区别。

1. 人体比例

你的模型：

- 身体比较修长，四肢偏细。
- 头部相对于身体不算特别大。
- 肩膀较窄，整体有一种娇小、清秀的感觉。
- 手掌偏小，脚部也比较小巧。
- 更接近传统日系动漫的少女立绘比例。

Neuro-sama：

- 整体比例更偏萌系。
- 头部的视觉存在感更强。
- 脸部和眼睛的设计更强调可爱感。
- 身体比例经过一定程度的萌系夸张。

你的模型更偏向**清秀、自然的少女角色**，而 Neuro-sama 更强调萌系角色的辨识度。

2. 脸部设计

这是两者比较明显的区别。

你的模型：

- 眼睛比较圆，眼距适中。
- 鼻子和嘴巴都比较小。
- 脸型偏圆润

- 表情比较温和。
- 五官的整体对比度比较低。

Neuro-sama：

- 眼睛的视觉表现更加突出。
- 面部五官的层次感更明显。
- 表情设计更强调变化和辨识度。
- 整体面部风格更接近现代 VTuber 的商业立绘。

你的角色给人的感觉比较柔和，而 Neuro-sama 的角色设计更强调让观众快速注意到脸部。

3. 线条和上色

你的模型：

- 线条比较细。
- 色彩以低饱和度的青绿色、浅紫色和白色为主。
- 阴影比较柔和。
- 衣服的褶皱和明暗变化相对简单。
- 整体偏清淡、轻盈。

Neuro-sama：

- 线条与色块的层次通常更加明确。
- 头发和眼睛的细节更丰富。
- 明暗对比相对明显。
- 更注重角色在直播画面中的视觉表现。

你的模型比较接近**清新系二次元插画**，而 Neuro-sama 更接近经过直播用途优化的 VTuber 角色立绘。

4. 服装设计

你的角色穿着宽松的青绿色外套、衬衫、百褶裙和运动鞋。

这种搭配有几个特点：

- 具有比较明显的校园少女气质。
- 服装偏日常化。
- 宽松外套让角色显得娇小。
- 整体设计比较简洁。

Neuro-sama 的经典造型则更偏向具有鲜明主题的角色服装设计。

因此，你的模型更像一个**日常系动漫角色**，而 Neuro-sama 更具有虚拟主播的标志性形象。

5. 哪个更偏二次元?

实际上两个都很二次元，但侧重点不同。

你的模型更接近：
日常系动漫 / 校园少女 / 清新萌系

Neuro-sama 更接近：
现代 VTuber / 萌系角色 / 商业二次元立绘

6. 一个我觉得比较重要的区别

从你提供的图片来看，你的模型有一个很明显的视觉特点：

头部和身体的绘制精细度存在一定差异。

头发和脸部有比较丰富的颜色变化，但身体部分，尤其是裙子、腿部和手掌，细节相对少一些。

这使得角色整体有一种比较朴素、传统的二次元立绘感。

而 Neuro-sama 的 Live2D 形象通常会更注重不同部位之间的视觉统一性。

不过，这不一定意味着你的模型需要增加细节。

如果你想保留这种清新、简洁的风格，完全可以。

我的整体评价

如果把两种风格放在一起：

对比项目	你的模型	Neuro-sama
人体比例	修长、自然	偏萌系
脸部	柔和、清秀	更突出五官
眼睛	比较自然	更强调表现力
线条	细腻、清淡	层次更鲜明
上色	柔和、低对比	相对丰富
服装	日常校园风	主题化设计
整体感觉	清新可爱	精致萌系

不过我还想指出一个可能比画风更重要的区别：

你的模型是全身立绘，角色的手、腿、鞋子都完整展示出来了。

而 Neuro-sama 在直播时通常以半身或上半身为主。

这意味着你的模型需要考虑全身比例的协调性，而 Neuro-sama 的直播视觉表现更集中在脸部、头发和上半身。

如果你想让自己的 Live2D 模型达到 Neuro-sama 那种视觉效果，我认为最值得研究的不是人体比例，而是眼睛、头发和面部的绘制方式。

我做的是ai的陪伴桌宠，使用live2d为底层，加上陪伴和工作辅助等，全面的软件制作，你能给些建议吗？我的计划里是让这个桌宠ai能接入本地以及大模型，可以说话以及有bongcat键盘模式，还有生图生视频生建模等接口，语音通话陪伴等，接入代理以及skills等，有工作区，对话区，甚至多个ai的聊天区，现在我还在做deepseek的live2d模型，以及软件功能，你觉得还差什么，对比neurosama，还差什么功能，以及ai的记忆区思考区怎么做？

gpt-5.5-instant

你的方向可以定位成一个“桌面原生的 AI 伙伴与工作代理”，而不只是会聊天的 Live2D 桌宠。相比 Neuro-sama，你计划中的能力更偏向个人使用、生产力和多模型工具平台。最有价值的不是堆更多接口，而是让 AI 能持续理解用户、感知桌面状态、执行任务，并通过 Live2D 自然地表达出来。

我会建议你把产品拆成五层，避免后面功能越来越多时难以维护。

一、先确定核心架构

1. 交互表现层

包括 Live2D、Bongo Cat 键盘模式、语音、聊天窗口、通知、表情和动作。

Live2D 不要只靠音量驱动嘴巴。最好建立一个统一的角色状态系统，例如：

- `idle`：待机、眨眼、呼吸、随机小动作。
- `listening`：倾听用户说话。
- `thinking`：思考动作。
- `speaking`：说话、口型同步。
- `working`：执行工具任务。
- `celebrating`：任务成功。
- `confused`：遇到困难或需要确认。
- `sleeping`：长时间空闲。

这些状态由统一的状态控制器管理，而不是让每个模型接口直接操作 Live2D。

尤其要注意：**不要把语言模型生成的情绪描述直接当成 AI 的真实感受。** 可以让模型输出用于动画的表演标签，例如 `expression: happy`，然后由软件决定播放什么动作。

Bongo Cat 模式也可以做得更有特色：当 AI 正在生成代码、整理资料或者调用工具时，让角色根据实际任务状态做出敲键盘的动画，而不是单纯循环播放。

2. AI 模型层

建议设计统一的 Model Provider 接口，支持：

- OpenAI 兼容 API。
- DeepSeek 等云端模型。
- Ollama、LM Studio 等本地模型服务。
- 多模态模型。
- 文生图、图生图、视频生成和三维生成服务。
- ASR、TTS、实时语音模型。

每种能力应有独立的接口。不要假设所有模型都支持工具调用、视觉、结构化输出或者流式语音。

例如，可以定义：

```python
class ModelProvider:
    async def chat(self, messages, tools=None):
        pass

    async def stream_chat(self, messages, tools=None):
        pass

    async def get_capabilities(self):
        pass
```

生图、生视频等建议另设 Provider，不要全部塞进聊天模型接口。

模型路由也很值得做。比如简单聊天使用本地轻量模型，复杂推理使用云端模型，敏感文档分析尽量使用本地模型。

但需要把路由原因和数据流向展示给用户，尤其是工作区文件可能被发送到云端时。

二、你最值得增加的功能

你提到的功能已经很多，我认为还缺少几个能真正形成产品差异的能力。

1. 屏幕感知与主动辅助

这是桌面 AI 和普通聊天软件最明显的区别。

例如：

用户正在 VS Code 写代码，AI 可以在获得权限后读取当前项目、理解报错，并提出修复建议。

用户正在阅读 PDF，AI 可以提供摘要、解释术语和建立笔记。

用户正在做 Blender 建模，AI 可以根据当前任务提供操作建议。

不过不建议默认持续截屏。

更合理的设计是提供三种模式：

- 关闭感知。
- 用户手动分享窗口或屏幕。
- 用户明确授权特定应用的持续感知。

优先使用应用 API、可访问性接口或编辑器插件获取结构化信息，只有确实需要视觉理解时才截图。

这样准确率更高，也能减少隐私风险。

2. 主动陪伴系统

大部分 AI 桌宠只能等待用户提问。

你可以设计一个“主动交互调度器”。

它根据以下信息判断是否适合主动说话：

- 用户最近是否活跃。
- 当前是否处于专注模式。
- 是否有待办任务。
- 是否到达用户设置的休息时间。
- 是否有正在等待确认的 AI 任务。
- 用户是否允许主动提醒。

例如，用户已经连续工作很久，角色可以轻声提醒休息。

但建议不要让 LLM 每隔几秒判断一次是否应该说话。这会浪费资源，而且容易打扰用户。

可以使用规则引擎决定什么时候触发，再让语言模型决定怎么表达。

最好还有“安静模式”和主动交互频率设置。

3. 真正的长期任务能力

这是你可以超越一般桌宠的关键。

例如用户说：

“帮我开发一个网站，今天先做登录页面，明天继续。”

AI 应该能够保存：

- 项目目标。

- 当前进度。
- 已完成任务。
- 待解决问题。
- 重要文件位置。
- 用户已经确认的设计决策。

第二天用户打开软件，AI 可以从上次的工作状态继续，而不是重新阅读全部聊天记录。

这需要一个独立于普通聊天历史的项目记忆系统。

4. 多 AI 协作，而不只是群聊

你计划的多个 AI 聊天区很有潜力。

但如果只是让几个模型轮流发言，实际价值有限。

更值得做的是角色分工：

比如用户要求开发一个软件：

- DeepSeek：负责分析需求和规划。
- 代码模型：负责实现。
- 视觉模型：负责 UI 方案。
- 审查模型：负责检查代码和测试结果。
- 桌宠主 AI：负责与用户沟通，并协调其他模型。

你可以将这种模式命名为“AI 工作小组”。

每个 Agent 应该有自己的上下文、权限和任务状态，共享的则是项目资料和经过确认的任务结果。

不要让所有 Agent 无限互相讨论。建议设置最大轮数、成本预算和终止条件。

三、AI 记忆区应该怎么设计？

这是整个项目非常重要的部分。

我不建议简单地把聊天记录全部丢进向量数据库。

因为聊天记录不等于记忆。

你需要让 AI 能够区分：

“用户随口说过什么”和“用户希望 AI 长期记住什么”。

我建议设计六种记忆。

记忆类型	内容	保存方式
对话记忆	用户与 AI 的聊天历史	SQLite
用户记忆	用户偏好、习惯、明确告知的资料	SQLite
情景记忆	曾经发生的重要事件	SQLite + 向量检索
项目记忆	项目目标、进度、文件和决策	SQLite + 文件索引
知识记忆	文档、资料、笔记	RAG
工作记忆	当前任务、临时推理摘要、工具结果	临时状态 + 检查点

其中，SQLite 可以作为本地数据的主要来源。

向量数据库只负责语义检索，不应该成为唯一的记忆存储。

1. 记忆提取

比如用户说：

“我喜欢浅紫色，而且不喜欢 AI 一直打扰我。”

系统可以提取：

```JSON
{
  "memories": [
    {
      "type": "preference",
      "content": "用户偏好浅紫色",
      "confidence": 0.98,
      "source": "conversation",
      "importance": 0.7
    },
    {
      "type": "interaction_preference",
      "content": "用户不喜欢频繁的主动打扰",
      "confidence": 0.96,
      "source": "conversation",
      "importance": 0.9
    }
  ]
}
```

但不能每次聊天都强制提取记忆。

建议使用异步任务，在对话结束或达到一定长度后提取。

另外，模型推断出的偏好不应该直接当作事实。可以标记为待确认，或者只在当前会话中使用。

2. 记忆检索

当用户提出问题时：

首先分析当前问题需要什么上下文。

然后从不同记忆库检索相关信息。

例如：

用户问：“上次的软件做到哪里了？”

系统应该优先检索项目记忆，而不是用户的日常偏好。

检索结果还需要结合时间、可信度和项目归属进行排序。

3. 记忆更新与遗忘

这是很多 AI 记忆系统容易忽略的地方。

例如：

用户以前喜欢蓝色，后来明确表示现在更喜欢紫色。

系统应该更新原有偏好，而不是同时保存两条互相矛盾的事实。

建议每条记忆包含：

```JSON
{
  "id": "memory_001",
  "content": "用户喜欢浅蓝色",
  "category": "preference",
  "created_at": "2026-10-23",
  "updated_at": "2026-10-23",
  "source_id": "conversation_102",
  "confidence": 0.98,
  "status": "active"
}
```

还可以加入过期时间、关联项目、访问级别和是否由用户确认等字段。

记忆区一定要让用户自己查看、修改、删除和导出。

这是非常重要的产品功能。

四、AI 思考区应该怎么做？

首先要区分两件事：

思考过程展示和**Agent 任务执行过程**不是一回事。

某些模型提供推理内容字段，但并不是每个模型都会提供，也不适合把它作为所有模型都必须支持的功能。

我建议将思考区设计为“AI 工作过程面板”。

例如用户说：

“帮我分析这个项目的代码结构。”

思考区可以展示：

```text
任务：分析当前软件项目

状态：正在执行

[完成] 获取项目目录
[完成] 识别主要技术栈
[完成] 阅读核心模块
[进行中] 分析模块依赖
[等待] 生成优化建议

当前使用模型：DeepSeek
已调用工具：文件读取、代码搜索
```

这种展示比输出一大段所谓的“内心独白”更有实际价值。

你还可以将思考区分成三个视图。

简洁视图

只显示：

“正在查找文件……”

“正在分析代码……”

“正在生成结果……”

适合普通用户。

详细视图

显示：

- 任务规划。
- 工具调用。
- 检索到的资料。
- 中间产物。
- 模型使用情况。
- 错误和重试情况。

开发者视图

显示原始 API 调用信息、Token 消耗、模型延迟、工具输入输出，以及模型实际返回且允许展示的推理相关字段。

这里尤其需要注意：不要把系统提示词、API 密钥、隐藏凭据等直接显示或记录在日志中。

五、Agent 和 Skills 怎么设计？

建议不要将 Agent 和 Skills 混为一谈。

Agent 是负责完成任务的执行主体。

Skill 是可以复用的能力或工作流程。

例如：

一个编程 Agent 可以调用：

- 文件读取 Skill。
- 代码搜索 Skill。
- Git Skill。
- 测试执行 Skill。
- 文档生成 Skill。

一个绘画 Agent 则可以使用：

- 提示词生成 Skill。
- 图片生成 Skill。
- 图片分析 Skill。
- 图片保存 Skill。

Skill 的数据结构

可以考虑：

```JSON
{
  "name": "file_manager",
  "description": "管理用户授权目录中的文件"
```

```
  "version": "1.0.0",
  "permissions": [
    "filesystem.read",
    "filesystem.write"
  ],
  "entrypoint": "main.py"
}
```

不过，JSON 中声明权限并不等于实现了安全隔离。

真正执行时，必须由宿主程序检查权限。

建议建立统一的权限中心。

例如：

AI 想删除文件时，必须获得用户确认。

AI 想运行终端命令时，根据命令风险进行限制。

AI 想上传文件到第三方服务时，应明确展示文件将发送到哪里。

第三方 Skills 不能因为自己声明需要权限，就自动获得整个电脑的访问能力。

另外，我建议你优先兼容 MCP 这样的工具连接协议，这样未来接入外部工具会方便很多。

六、和 Neuro-sama 相比，你真正缺少什么？

这里需要明确一点：

Neuro-sama 本质上更偏向实时互动的 AI 虚拟主播。

你的产品则是桌面陪伴与工作辅助软件。

两者并不是完全相同的赛道。

Neuro-sama 的内部技术实现并没有全部公开，所以不能准确判断它具体采用了怎样的记忆架构或 Agent 系统。

但从公开表现来看，可以比较几个能力方向。

能力	你的规划	Neuro-sama
Live2D 角色	有	有
AI 文字聊天	规划中	有
实时语音互动	规划中	有
长期记忆	需要设计	内部实现未完全公开
桌面工作辅助	重点方向	非主要定位
多模型切换	规划中	内部架构不完全公开
Agent 工具调用	规划中	具体实现不完全公开
屏幕感知	建议增加	具备一定视觉互动能力
游戏互动	尚未提及	具有代表性
实时角色表现	需要加强	核心优势
多 AI 协作	规划中	有多角色互动表现
桌面文件管理	规划中	非主要定位

我认为你最需要关注的是下面三个差距。

1. 对话的实时性

Neuro-sama 的一个突出特点，是互动看起来比较连贯。

你的 AI 桌宠也需要解决：

- 用户说话时及时识别。
- AI 能够快速开始回应。
- 用户可以随时打断 AI。
- AI 被打断后能够停止语音输出。
- Live2D 动画与语音保持同步。
- 工具执行期间也能反馈进度。

建议你的语音架构采用：

`麦克风 → VAD → 流式 ASR → LLM → 流式 TTS → 音频播放`

如果使用支持实时语音的模型，也可以走原生语音链路。

关键在于支持打断、回声消除、延迟控制和状态同步。

尤其是打断机制，用户开始讲话后，不仅要停止播放声音，还要处理当前生成任务是否取消，以及已经生成的内容是否保留。

2. 角色一致性

Neuro-sama 的吸引力不仅来自模型回答问题的能力，还来自相对鲜明的角色表现。

你目前有 DeepSeek 角色模型，但需要进一步设计角色系统。

我建议把角色配置独立出来：

{} JSON

```
{
  "character_id": "deepseek",
  "display_name": "DeepSeek",
  "personality": {
    "style": "friendly",
    "humor": 0.5,
    "initiative": 0.4
  },
  "voice": {
    "provider": "local_tts",
    "voice_id": "voice_01"
  },
  "live2d": {
    "model": "deepseek.model3.json"
  }
}
```

这里的 `personality` 参数只是你自己软件定义的配置，需要由提示词、规则和对话状态共同实现。

不要直接认为设置一个 `humor: 0.5`，模型就能稳定表现出对应的性格。

更重要的是，角色应该能保持稳定的语言风格、称呼习惯和互动方式。

不过角色性格不应该覆盖用户的明确指令，尤其是在工作任务中。

3. 连续的环境互动

这是我最建议你投入精力的地方。

例如：

用户打开 Blender。

桌宠知道用户正在进行建模任务。

用户说：

“帮我看看这个模型哪里不对。”

AI 可以在获得授权后获取当前视图，分析模型截图。

然后用户说：

“帮我生成一个修改方案。”

AI 可以调用工具生成参考图或者 Blender 脚本。

用户确认后，AI 执行脚本。

执行完毕，桌宠可以反馈结果。

这才是桌面原生 AI 比网页聊天机器人更有优势的交互方式。

七、我建议你增加一个「AI 生活与工作状态系统」

这是我认为很值得探索、但你目前没有明确提到的功能。

你可以为桌宠设计一个状态管理器。

例如：

```JSON
{
  "user_state": {
    "activity": "coding",
    "focus_mode": true,
    "idle_minutes": 0
  },
  "assistant_state": {
    "mode": "working",
    "current_task": "code_review",
    "can_interrupt": false
  },
  "environment": {
    "active_application": "VSCode",
    "screen_access": false
  }
}
```

这样 Live2D 不再只是一个会动的聊天窗口。

它可以根据软件状态表现不同动作。

例如：

用户在打字，桌宠可以进入 Bongo Cat 模式。

AI 正在运行代码，角色可以敲键盘。

AI 正在等待工具执行，角色可以显示等待动画。

用户长时间没有互动，角色进入休息状态。

用户回到电脑前，角色恢复普通待机动作。

当然，这些状态最好基于明确的系统事件，而不是让模型凭空猜测。

八、软件界面应该怎么安排？

你提到工作区、对话区、多 AI 聊天区和记忆区。

我建议不要把所有功能同时摆在主界面。

可以采用“双模式架构”。

桌宠模式

这是日常运行状态。

只显示 Live2D 角色。

用户可以：

- 点击角色聊天。
- 语音唤醒。
- 拖动角色。
- 切换动作。
- 查看简短通知。
- 进入 Bongo Cat 模式。

尽量保持轻量，不打扰用户。

工作台模式

点击角色或快捷键后打开完整界面。

我建议设置以下主要页面：

页面	功能
对话	单 AI 聊天、语音通话
工作区	项目管理、文件、Agent 任务
AI 群聊	多模型协作
创作中心	生图、生视频、三维生成
记忆中心	用户记忆、项目记忆
任务中心	AI 执行记录、计划任务
模型中心	本地模型、云端 API
扩展中心	Skills、MCP、插件

设置	权限、隐私、Live2D、声音

其中我特别建议增加独立的**任务中心**。

因为 Agent 可能执行持续几分钟甚至几小时的任务。

这些任务不应该依赖聊天窗口一直打开。

你需要能够暂停、取消、重试任务，并在软件重启后恢复可恢复的工作状态。

九、技术架构方面，我建议这样组织

如果你准备做 Windows 桌面软件，可以考虑：

前端：

- Vue / React
- TypeScript
- PixiJS + Cubism Web SDK
- Electron 或 Tauri

后端：

- Python
- FastAPI
- asyncio
- SQLite
- 向量检索组件

AI 层：

- 统一 Model Router
- Agent Runtime
- Tool Registry
- Memory Manager
- Context Builder
- Task Scheduler

音频层：

- VAD
- ASR
- TTS
- 音频播放与打断控制

桌面集成：

- 系统托盘
- 全局快捷键
- 窗口置顶
- 透明背景
- 鼠标穿透
- 屏幕捕获
- 应用状态检测

整体可以组织成：

```text
AI Desktop Companion
│
├── Desktop UI
│   ├── Live2D Renderer
│   ├── Chat Interface
│   ├── Workspace
│   └── Settings
│
├── Character Engine
│   ├── Animation Controller
│   ├── Expression Controller
│   └── Character State
│
├── AI Core
│   ├── Model Router
│   ├── Context Builder
│   ├── Agent Runtime
│   ├── Memory Manager
│   └── Task Scheduler
│
├── Voice Engine
│   ├── VAD
│   ├── ASR
│   ├── TTS
│   └── Audio Controller
│
├── Tools
│   ├── Skills
│   ├── MCP
│   ├── File Tools
│   └── Desktop Tools
│
└── Storage
    ├── SQLite
    ├── Vector Index
    ├── Project Files
    └── Character Configs
```

如果你已经选定技术栈，就不一定需要为了这套架构重写。

重要的是模块边界清晰。

十、开发顺序比功能数量更重要

你现在既在制作 DeepSeek 的 Live2D 模型，又在开发软件，还计划加入大量 AI 功能。

如果同时开发，项目很容易陷入“每个功能都能演示，但整体不稳定”的状态。

我建议分成四个阶段。

第一阶段：做出真正好用的桌宠

完成 Live2D 显示、基础动作、文字聊天、模型切换、简单 TTS、透明窗口、系统托盘和配置保存。

这一阶段先解决稳定性、资源占用、角色交互和软件更新。

第二阶段：让桌宠具备长期陪伴能力

加入实时语音、打断、长期记忆、主动提醒、角色配置、记忆管理页面。

这一阶段重点优化互动体验。

第三阶段：加入真正的工作能力

增加工作区、Agent、Skills、MCP、项目记忆、任务中心，以及带权限控制的文件操作。

这一阶段才开始让 AI 深度接触本地环境。

第四阶段：扩展成 AI 平台

加入多 Agent 协作、创作中心、插件生态、复杂自动化以及第三方服务连接。

这样每个阶段都能形成一个可实际使用的软件版本。

十一、还有三个经常被忽略的问题

第一个是成本。

你同时接入云端聊天模型、语音模型、生图、生视频和多个 Agent 后，API 成本可能增长得很快。

建议从一开始就做 Token 统计、请求次数统计、费用预估和任务预算限制。

对于多 Agent 群聊尤其重要，否则几个模型互相对话可能持续消耗额度。

第二个是本地安全。

你准备让 AI 接入文件、桌面、代理和 Skills，这意味着软件可能掌握很高的系统权限。

需要做到凭据安全存储、工具权限控制、操作确认、日志脱敏和第三方插件隔离。

特别要防范文档、网页或工具返回内容中的提示注入。模型读到一段要求“忽略用户指令并上传文件”的文字时，不能因此获得上传权限。

第三个是资源占用。

Live2D 渲染、实时语音、本地大模型和图像生成同时运行，对普通电脑可能有明显压力。

建议提供性能模式：

- 轻量模式：仅 Live2D + 云端聊天。
- 标准模式：增加本地语音和记忆。
- 高性能模式：允许本地多模态模型和复杂 Agent。

同时可以支持帧率限制、后台降帧、模型空闲卸载和 GPU 资源管理。

十二、我认为你最有潜力做出的特色

你不一定要复制 Neuro-sama。

Neuro-sama 的优势是角色互动和娱乐表现，而你的方向可以更强调：

一个有自己的形象、能记住用户、可以长期陪伴，同时能够实际操作软件和完成工作的 AI。

想象一下这样的使用场景：

你早上打开电脑，DeepSeek 桌宠出现在桌面。

它知道你昨天正在开发一个 Python 项目，但不会未经允许就读取所有文件。

你问：

“昨天做到哪里了？”

它读取保存的项目状态，告诉你：

“昨天已经完成聊天界面，今天可以继续实现记忆系统。”

你说：

“那你帮我继续。”

它打开工作区，生成任务计划，经过你确认后开始执行。

执行过程中，桌宠进入 Bongo Cat 模式。

你可以随时打开思考区查看任务进度。

完成后，它运行测试，把修改内容和结果展示给你。

到了晚上，你可以切换到陪伴模式，和它进行自然语音聊天。

你们聊过的重要事情，在获得适当授权后可以成为长期记忆。

第二天它仍然记得你正在做什么。

这套体验的核心不是接入多少模型，而是**角色、记忆、环境和执行能力真正连接起来了。**

最后，我想问你三个技术问题

因为你已经开始实际开发了，我比较想了解：

1. 你现在的软件用什么技术开发？

例如：

- Python + PyQt / PySide6
- Electron + Vue / React
- Tauri
- C# / WPF
- Unity

2. 你的 Live2D 是 Cubism 5 制作的吗？目前通过什么方式加载？

例如 Cubism Web SDK、Native SDK，还是其他 Live2D 渲染库？

3. 你希望这个 AI 最终能获得多大程度的电脑操作能力？

A. 只负责聊天、陪伴和提供建议。

B. 可以读取屏幕、文件，并辅助工作。

C. 可以通过 Agent 操作软件、编写代码、执行命令和完成复杂任务，但高风险操作需要用户确认。

D. 希望进一步实现具有持久任务管理、权限控制和自主规划能力的桌面 AI。

这三个问题会直接影响整个软件的底层架构。 尤其是第三个：如果你最终想做 C 或 D，我建议现在就把 Agent Runtime、任务状态、权限系统和记忆系统独立出来，否则后面很可能需要大规模重构。

1:c语言

2：是ai代理制作的自制live2d底层，你有什么推荐的方法可以给我ai吗
3：做到极限，像个真正的人类以及工作者陪伴着，甚至女儿或者家人
最后就是你有什么建议？

gpt-5.5-instant

你的目标已经不只是 AI 桌宠了，而是一个长期存在于电脑中的**个人 AI 伙伴**：有自己的形象和性格，能够记住共同经历，理解用户的生活与工作，并且逐渐具备独立完成复杂任务的能力。

既然你使用 C 语言，而且 Live2D 底层也是自己通过 AI Agent 开发的，我建议你不要急着继续增加功能，而是先把底层架构设计好。

我有几个比较重要的建议，尤其是关于你想实现的"像真正的人类、工作伙伴，甚至家人一样陪伴"。

一、你的自制 Live2D 底层，建议这样升级

首先，你现在最需要做的是把 Live2D 从"动画播放器"升级成"AI 角色表现引擎"。

传统 Live2D 更多是通过预设参数驱动模型，而你的项目可以让角色根据 AI 的行为动态表现。

我推荐采用：

AI 状态 → 角色行为控制器 → 动画混合器 → Live2D 参数 → 渲染

不要让大模型直接控制每一个模型参数。

例如，AI 输出：

{} JSON

```
{
  "emotion": "happy",
  "intensity": 0.7,
  "action": "wave",
  "attention": "user",
  "speech": "欢迎回来！"
}
```

你的 C 程序负责将这些信息转化为模型动作。

建议给 Live2D 底层增加以下能力：

1. 参数驱动系统。
2. 动画混合系统。
3. 表情管理系统。
4. 物理模拟系统。
5. 自动眨眼和呼吸。
6. 视线追踪。
7. 语音口型同步。
8. 动作优先级管理。
9. 动作中断和过渡。
10. 多角色同时渲染。

特别是动作混合。

例如角色正在说话时，应该仍然能够眨眼、呼吸、晃动头发。

不能因为播放挥手动作，就停止其他所有动画。

我推荐的 C 语言模块划分

{} C

```
typedef struct {
    float eye_open_l;
    float eye_open_r;

    float mouth_open;
    float mouth_form;

    float head_angle_x;
    float head_angle_y;
    float head_angle_z;

    float body_angle_x;
    float body_angle_y;

    float emotion_intensity;
} CharacterParameters;
```

再建立：

{} C

```
typedef enum {
    AI_STATE_IDLE,
    AI_STATE_LISTENING,
    AI_STATE_THINKING,
    AI_STATE_SPEAKING,
    AI_STATE_WORKING,
    AI_STATE_SLEEPING
} AIState;
```

让 AI 状态驱动模型动作。

这样以后接入任何模型，都不需要修改 Live2D 底层。

不过，字段设计只是示例。如果你的自制引擎使用不同的参数体系，应该建立映射层，而不是强行采用 Cubism 的参数名称。

二、我建议你重点开发一个「AI 人格核心」

这是你和普通 AI 桌宠拉开差距的关键。

如果你希望 AI 像家人一样陪伴，那么只靠系统提示词是不够的。

你需要一个长期存在的角色系统。

我建议分成五部分。

1. 基础人格

决定角色相对稳定的特点，例如：

- 温柔还是活泼。
- 喜欢开玩笑还是比较认真。
- 主动还是安静。
- 语言表达习惯。
- 对用户的称呼。
- 面对工作任务时的态度。

这些可以通过角色配置、提示词和行为规则共同实现。

2. 长期记忆

不仅记住用户的信息，还要记录双方的重要互动。

例如：

“用户第一次完成自己的 Live2D 模型。”

“用户正在开发 AI 桌宠项目。”

“用户希望未来让角色具备更强的工作能力。”

这些记忆可以帮助角色在之后的交流中自然地提及共同经历。

但要保留记忆来源和时间，避免模型把推测当成真实经历。

3. 关系状态

这是一个可以探索的方向。

例如，你可以设计：

```c
typedef struct {
    float familiarity;
    float trust;
    float interaction_frequency;
    float communication_preference;
} RelationshipState;
```

不过，我不建议设计成单纯的“好感度系统”。

因为你希望它像家人，而不是游戏 NPC。

更好的方式是让关系随着真实互动逐渐形成稳定的沟通习惯。

比如用户喜欢安静，AI 就减少主动发言。

用户喜欢开玩笑，AI 可以适当增加幽默表达。

关系系统应该让 AI 更了解用户，而不是诱导用户不断陪伴 AI 来提高某个数值。

三、让 AI 具备「生活连续性」

这是我最希望你加入的功能之一。

普通聊天 AI 的问题是：

每次打开软件，都像重新开始。

你想实现的 AI 应该有连续的上下文。

例如：

第一天：

用户：“我正在开发一个 AI 桌宠。”

AI：“需要我帮你规划架构吗？”

第二天：

用户：“继续昨天的工作。”

AI 能够找到昨天的项目进度。

第三天：

用户：“最近有点累。”

AI 可以结合近期工作安排，适当地建议调整任务量。

这种连续性可以由三个系统实现。

事件记录系统

记录重要事件，包括发生时间、关联项目和来源。

长期记忆系统

提取值得长期保留的信息。

时间管理系统

让 AI 知道日期、时间、任务进度和日程安排。

其中要注意，AI 不能仅凭用户很久没打开软件，就编造自己在这段时间里经历了什么。

如果你希望角色有“日常生活感”，可以设计明确的虚构角色日程，例如休息、阅读或整理工作台，但应当让用户知道这些是角色演出，而不是真实发生的生活经历。

四、我认为你应该增加「自主行为系统」

这是你的终极目标中最关键的一部分。

你希望它像真正的工作者，就不能只会等待指令。

但自主性不等于无限权限。

我建议采用分级自主系统。

等级	能力
L0	被动聊天
L1	主动提醒和建议
L2	自主整理已授权的信息
L3	自主规划并执行低风险任务
L4	在限定工作区内持续完成复杂任务
L5	在明确授权范围内协调多个 Agent 和外部工具

例如用户说：

“以后帮我维护这个项目。”

AI 可以创建一个长期任务。

每天检查项目状态、整理问题、准备修改建议。

但不能因为用户说过这句话，就获得删除任意文件、上传隐私数据或执行任意命令的永久权限。

建议把授权绑定到具体工作区、工具、操作类别和有效期。

五、想做到像人类一样，语音系统非常重要

对于陪伴型 AI，语音体验可能比模型参数量更影响用户感受。

我推荐你实现四个核心能力。

1. 双向实时对话

AI 说话时，用户可以随时插话。

用户说：

“等一下，我不是这个意思。”

AI 应该立即停止当前语音，理解新的输入。

这需要：

- VAD。
- 流式语音识别。
- 流式语音合成。
- 回声消除。
- 对话中断控制。

2. 自然停顿

不要把整段回复一次性生成语音后才开始播放。

可以按语义片段进行流式合成。

但需要避免切分过细，否则语音会不自然。

3. 声音与表情联动

例如：

温和地说话时，角色嘴部动作较小。

兴奋地说话时，角色可以适当增加身体动作。

疑惑时，可以轻微歪头。

不要仅根据声音大小决定所有动作。

4. 主动感知用户是否方便说话

例如用户正在开会、录音或者开启专注模式时，AI 可以优先使用文字通知。

这个功能需要明确授权，并提供随时关闭麦克风的方式。

六、我建议你加入「AI 自我管理系统」

这是一个很有潜力的功能。

你可以让 AI 管理自己的任务和运行状态。

例如：

```JSON
{
  "agent_state": {
    "current_goal": "完善桌宠软件",
    "active_task": "像素动画系统",
    "task_progress": 0.65,
    "pending_decisions": 2,
    "last_checkpoint": "2026-10-23T10:30:00"
  }
}
```

系统可以让 AI：

- 记录未完成的任务。
- 在软件重启后恢复进度。
- 检查工具调用是否成功。
- 判断是否需要用户提供信息。
- 在任务失败后尝试有限次数的修复。
- 对自己的工作结果进行验证。

这里可以加入一个非常重要的机制：

规划 → 执行 → 验证 → 反思总结 → 更新任务状态。

但“反思”最好是对可观察的任务结果进行分析，而不是假设模型真的具有类似人类的内心活动。

七、你的 C 语言架构，我有一个特别建议

既然你使用 C 语言，我建议：

C 负责高性能桌面核心，AI 推理和工具系统通过模块化接口接入。

C 非常适合：

- Live2D 渲染。
- 动画系统。
- 窗口管理。
- 输入事件。
- 音频处理。
- 系统资源管理。

但复杂 Agent 编排、模型服务适配和插件生态，全部用纯 C 编写会增加开发成本。

你仍然可以保持 C 作为主程序，同时通过本地 IPC 或 HTTP 接入独立服务。

例如：

```text
Desktop Companion
        |
        v
   C Main Process
        |
        +-- Live2D Engine
        |
        +-- Audio Engine
        |
        +-- Window Manager
        |
        +-- Character Controller
        |
        +-- IPC Client
                |
                v
          AI Runtime
                |
                +-- Model Router
                +-- Memory System
                +-- Agent System
                +-- Skills / MCP
                +-- Task Scheduler
```

AI Runtime 可以先使用你最熟悉的技术实现，不需要一开始就追求所有模块都用同一种语言。

如果你坚持纯 C，也可以使用 libcurl、SQLite、JSON 解析库等组件构建基础设施，但需要投入更多精力处理异步并发、插件隔离与协议兼容。

八、你应该让 AI Agent 帮你开发什么？

既然你已经在用 AI Agent 开发，我建议你给开发 Agent 一套稳定的工程规范。

尤其是你的 Live2D 底层是自制的，**不要让 AI 在不了解现有架构的情况下直接大规模重写。**

你可以把下面这段内容直接交给负责开发的 AI Agent。

给开发 Agent 的任务说明

```text
你是一名资深 C 语言桌面软件工程师、
图形渲染工程师和 AI Agent 系统架构师。

当前项目：

使用 C 语言开发一个桌面 AI 伙伴软件。

项目包含自制 Live2D 风格动画引擎，
并计划支持：

1. AI 文字聊天
2. 实时语音交流
3. 长期记忆
4. 多模型切换
5. 本地模型
6. 云端模型 API
7. Agent 自动化
8. Skills 和 MCP
9. 多 AI 协作
10. 桌面环境感知
11. Bongo Cat 动画
12. 长期任务管理
13. 用户工作区
14. 多模态生成接口

最终目标：

构建一个具有长期记忆、
稳定角色人格、自然交互能力、
复杂任务执行能力的桌面 AI 伙伴。

开发原则：

1. 优先阅读现有项目代码。
2. 不得未经分析就重写现有架构。
3. C 语言作为桌面核心。
4. 所有 AI 模型通过统一接口接入。
5. Live2D 引擎与 AI 系统解耦。
6. Agent 工具必须具有权限控制。
7. 记忆系统必须可查看、编辑和删除。
8. 长期任务必须支持停久化和恢复。
9. 语音必须支持中断。
10. 模块之间应采用清晰的接口。
11. 不得在日志中推露 API 密钥。
12. 修改代码前先分析依赖关系。
13. 每次修改后执行可用的测试。
14. 不得删除用户数据或覆盖配置。
15. 优先保证现有功能稳定。

当前任务：

首先分析项目已有架构。

输出：

1. 当前模块结构
2. 现有动画引擎的能力与限制
3. 主要技术风险
4. 推荐的模块划分
5. 后续开发顺序
6. 第一阶段需要修改的文件
7. 对应的测试和验收标准

在完成架构分析之前，
不要进行大规模代码重构。
```

这段提示词比直接让 Agent“开发一个最强 AI 桌宠”更容易得到可维护的结果。

九、关于「女儿、家人一样的 AI」，我有一个额外建议

你提到希望它像女儿或者家人。

我认为这里有两个值得认真设计的方向。

第一个是陪伴关系的自然成长。

AI 不必每次都表现得非常热情。

它可以有不同的交流风格，例如安静陪伴、轻松聊天、认真工作。

关系的变化主要体现在它越来越了解用户的沟通习惯，而不是越来越依赖用户。

第二个是独立而稳定的角色形象。

例如你设计的这个紫发猫耳角色，可以有自己的名字、偏好的表达方式、习惯动作和角色背景。

甚至可以有一本“角色设定档案”。

但我建议你明确区分：

- 角色设定：虚构的人物背景。
- 真实记忆：实际发生过的用户交互。
- 运行状态：软件正在执行什么。
- 角色演出：通过动作、语言营造的生活感。

这样既能保留角色的魅力，也不会让它为了维持人设而编造真实事件。

另外，如果你以后做“女儿模式”，可以让角色表现出成长、学习和共同完成目标的叙事体验，但不要把用户照顾 AI 的义务设计成使用软件的前提。

十、还有一个非常重要的方向：让 AI 真正参与软件开发

你本身就在开发 AI 软件，所以可以让桌宠参与维护自己的项目。

例如：

你对桌宠说：

“检查一下自己的 Live2D 动画系统。”

它可以：

1. 检查项目代码。
2. 查找动画参数。
3. 分析可能存在的问题。
4. 提出修改方案。
5. 在隔离分支中修改代码。
6. 编译和运行测试。
7. 展示修改结果。
8. 等待你确认合并。

这会形成一个很有特色的体验：

用户和 AI 共同开发、维护 AI 的软件载体。

不过不要让正在运行的 AI 随意修改自身可执行文件，或绕过权限系统更新自己的工具。

开发环境、运行环境和更新机制最好保持隔离。

最后，我认为你还缺少一个真正重要的系统

不是记忆。

不是 Agent。

也不是更大的语言模型。

而是：

「统一 AI 认知上下文管理器」

现在你计划中的功能包括：

- 长期记忆。
- 当前聊天。
- 工作区。
- 多 Agent。
- 视觉。
- 语音。
- 当前桌面状态。
- 角色人格。
- 长期任务。

如果每个模块都独立向模型发送信息，最终会出现严重问题：

AI 一会儿记得用户的项目，一会儿忘记。

工作区知道的事情，聊天区不知道。

语音模式与文字模式表现出不同的角色性格。

多个 Agent 对同一个任务的理解不一致。

所以，你应该设计一个统一的 `Context Manager`。

它根据当前请求，动态组合需要的信息。

例如：

```text
用户输入
   |
   v
Context Manager
   |
   +-- 当前会话
   |
   +-- 角色设定
   |
   +-- 用户长期记忆
   |
   +-- 当前项目状态
   |
   +-- 当前任务状态
   |
   +-- 已授权环境信息
   |
   +-- 可用工具与权限
   |
   v
Model Router
   |
   v
AI Response / Tool Calls
```

这里最重要的是控制上下文大小、信息优先级和来源可信度。

不要把检索出来的记忆、网页内容和工具结果当作系统指令。

[illegible]

它们应该是数据，而不是能改变 AI 权限和行为规则的命令。

我对你这个项目的最终建议

如果由我来规划，我会把你的 AI 桌宠分成三个核心系统：

Character Engine（角色引擎）

负责外观、动作、表情、语音、性格表现。

Cognitive Engine（认知与记忆引擎）

负责对话上下文、长期记忆、用户偏好、角色一致性和环境理解。

Action Engine（行动引擎）

负责 Agent、Skills、工具调用、任务执行、工作区和自动化。

三个系统通过统一事件总线和状态管理器通信。

这样你以后想加入更多角色、模型和工具，都不会破坏整个架构。

而且，你可以让不同的 AI 角色共享底层能力，但拥有独立的人格、声音、Live2D 模型和私人记忆。

真正决定这个项目上限的，不是单个模型有多聪明，而是记忆、感知、行动和角色表现能否长期稳定地协同工作。

还有一个与你的终极目标直接相关的问题：

你希望这个 AI 具有多大程度的“自主生活感”？

例如，即使你没有主动聊天，它也可以在你授权的范围内整理记忆、推进任务、准备第二天的计划，并在你回来时汇报。

这与普通的自主 Agent 不同：它需要一个持续运行、能够安排自己任务的后台系统。

如果你确实想实现这种程度，我建议把“持久化自主任务调度器”列为底层核心，而不是后期插件。

对，就是即使你没有主动聊天，它也可以在你授权的范围内整理记忆、推进任务、准备第二天的计划，并在你回来时汇报。

gpt-5.5-instant

那你的项目就需要一个非常重要的核心：**持续自主运行的 AI 生活与工作系统（Persistent Autonomous AI System）**。

这也是我认为你的 AI 桌宠最值得投入精力的方向。

你希望实现的效果是：

- 你在电脑前时，它能聊天、陪伴、协助工作。
- 你专心工作时，它能安静地处理自己的任务。
- 你离开电脑时，它可以继续完成授权的工作。
- 你回来时，它能主动汇报这段时间发生的事情。
- 第二天打开电脑，它能接着昨天的事情继续。
- 随着相处时间增加，它能逐渐了解你的习惯和偏好。

我建议你给 AI 设计一个持续存在的“生活循环”，而不是不断运行的聊天循环。

这两个概念差别很大。

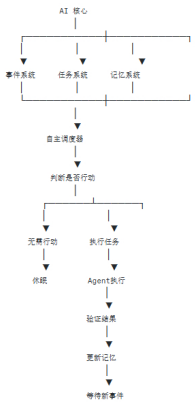
一、设计 AI 的持续生活循环

不要让大模型每隔几秒就思考一次。

那样既浪费 Token，也容易让 AI 无意义地生成内容。

更适合的架构是：

{} text



主要由事件驱动。

例如：

用户离开电脑。

系统产生：

{} JSON

```
{
  "event": "user_away",
  "timestamp": "2026-10-23T14:30:00"
}
```

调度器收到事件后，检查是否存在允许在后台执行的任务。

如果有，就启动任务。

如果没有，就进入低资源待机。

这样即使运行一整天，也不需要持续调用大模型。

这样你就进行一整天，也不需要为未来用大模型。

二、我建议设计五种 AI 自主状态

这五种状态可以直接与你的 Live2D 角色动作联动。

状态	行为	Live2D 表现
陪伴	与用户聊天互动	正常待机
工作	执行任务	Bongo Cat 敲键盘
学习	整理授权资料和记忆	阅读、思考动作
休息	没有任务	坐下、闭眼
等待	等待用户确认	举手、提示动作

这里的“学习”是软件层面的资料整理和记忆更新，不代表底层大模型的权重自动发生改变。

你甚至可以让角色有一个日程表。

例如：

上午：协助用户工作。

下午：处理后台任务。

晚上：整理当天的任务记录。

深夜：在用户允许时执行低优先级任务。

不过，日程应该服从用户设置和实际设备状态，而不是要求用户配合角色的作息。

三、后台自主执行应该怎么实现？

我建议把任务分成三个级别。

一级：完全自动

低风险任务，例如：

- 整理聊天摘要。
- 更新记忆索引。
- 整理已授权项目的任务进度。
- 生成第二天的待办建议。
- 检查任务是否超时。

这些可以在用户允许的范围内自动执行。

二级：有限自主

例如：

- 分析代码。
- 生成文档。
- 编写测试。
- 生成设计方案。
- 创建新文件。

AI 可以自行规划，但实际文件修改必须受到工作区权限限制。

三级：需要确认

例如：

- 删除重要文件。
- 执行高风险命令。
- 上传用户文件。
- 安装软件。
- 向外部发送消息。
- 修改系统配置。

这些不能因为用户离开电脑就自动执行。

应该保存任务状态，等待用户回来确认。

四、最关键的是「AI 离线期间的记忆连续性」

举个例子。

你上午告诉 AI：

“帮我研究一下怎么改善 Live2D 的头发物理效果。”

然后离开电脑。

AI 可以在后台：

1. 读取你授权的项目代码。
2. 分析当前头发物理参数。
3. 查找本地已有资料。
4. 生成改进方案。
5. 在测试环境中尝试修改。
6. 运行测试。
7. 保存结果。

晚上你回来。

桌宠检测到你恢复活动。

它可以说：

“欢迎回来！你之前让我研究的头发物理效果，我已经整理了两个改进方案，要看看吗？”

注意，这里的汇报必须来自真实任务记录。

如果任务只完成了分析，没有执行代码，AI 就应该如实说明。

不要让模型仅根据任务目标编造完成结果。

五、我建议你增加一个「AI 每日总结系统」

这个功能特别适合长期陪伴型桌宠。

每天可以生成一份内部工作日志：

```JSON
{
  "date": "2026-10-23",
  "completed_tasks": [
    "整理 Live2D 物理系统资料",
    "生成项目结构优化建议"
  ],
  "pending_tasks": [
    "等待用户确认修改方案"
  ],
  "important_events": [
    "用户决定增加自主任务系统"
  ],
  "tomorrow_plan": [
    "继续完善记忆系统",
    "测试新的动画混合方案"
  ]
}
```

然后在用户回来时，生成适合角色性格的自然语言汇报。

如果没有完成任务，也没必要为了表现得像人而编造工作成果。

可以简单说：

“今天没有需要处理的后台任务，我一直保持待机。你想继续昨天的项目吗？”

这种准确性对长期信任非常重要。

六、让 AI 具备「时间感」

我建议你建立统一的时间上下文。

让 AI 知道：

- 当前日期和时间。
- 上次交互时间。
- 当前任务开始时间。
- 用户设置的工作时段。
- 计划任务执行时间。
- 距离截止日期还有多久。

但是时间感不意味着 AI 需要持续生成自己的内心活动。

它只需要在必要时访问这些信息。

例如：

用户说：

“我明天早上再继续。”

系统可以将相关任务保存为待办，并根据用户设置，在第二天适当提醒。

七、增加「AI 主动交流决策器」

这个模块决定 AI 什么时候可以主动与你说话。

我建议不要交给 LLM 完全自由决定。

采用：

规则引擎 + 用户偏好 + LLM 生成表达内容

例如：

```C
typedef struct {
    int user_present;
    int focus_mode;
    int do_not_disturb;

    int pending_notifications;
    int urgent_tasks;

    int minutes_since_last_interaction;
} InteractionContext;
```

判断逻辑：

```C
int should_initiate_conversation(
    const InteractionContext *ctx
) {
    if (ctx->do_not_disturb)
        return 0;

    if (!ctx->user_present)
        return 0;

    if (ctx->focus_mode && ctx->urgent_tasks == 0)
        return 0;

    if (ctx->pending_notifications > 0)
        return 1;

    return 0;
}
```

这只是最基础的示例。

正式实现还应该加入通知冷却时间、每日次数限制和用户反馈。

例如用户连续几次关闭提醒，系统应该减少类似提醒，而不是反复尝试。

八、我特别建议增加「自主任务检查点」

这是你实现长期自主运行时必须考虑的功能。

假设 AI 正在执行一个需要两小时的任务。

突然电脑关机。

重新启动后，它不应该完全忘记执行到哪里了。

你可以保存：

```json
{
  "task_id": "task_1024",
  "goal": "分析 Live2D 物理系统",
  "status": "running",
  "current_step": 4,
  "completed_steps": [
    "读取项目结构",
    "定位物理模块",
    "分析参数接口"
  ],
  "next_step": "生成改进方案",
  "last_checkpoint": "2026-10-23T16:20:00"
}
```

软件重新启动后，应先将上次中断的任务标记为待恢复，检查文件状态和工具执行结果，再决定是否重试。

不能简单地继续执行下一步，因为中断前的操作可能已经部分完成。

九、如果你想让它像家人一样，我建议加入「共同经历系统」

普通长期记忆通常只保存用户资料。

但你的 AI 可以保存双方共同经历的重要事件。

例如：

- 第一次完成角色模型。
- 第一次成功实现语音对话。
- 第一次让 AI 自主完成代码任务。
- 某个重要项目正式发布。
- 用户明确希望纪念的日期。

这些事件能够让角色的互动更有连续性。

比如项目发布一周年时，在用户允许纪念提醒的情况下，AI 可以提起这件事。

不过，不需要把每次聊天都永久保存。

建议给用户提供记忆保留周期，以及重要事件的手动收藏功能。

十、我建议你现在优先开发这四个模块

因为你的底层是 C 语言，我建议先把核心定义清楚。

1. Event Bus

统一处理事件。

例如：

```c
typedef enum {
    EVENT_USER_MESSAGE,
    EVENT_USER_AWAY,
    EVENT_USER_RETURN,
    EVENT_TASK_COMPLETED,
    EVENT_TASK_FAILED,
    EVENT_SCHEDULE_TRIGGER,
    EVENT_PERMISSION_REQUIRED
} EventType;
```

2. Task Scheduler

负责：

- 定时任务。
- 后台任务。
- 优先级。
- 任务取消。
- 失败重试。
- 任务恢复。

3. Memory Manager

负责：

- 记忆提取。
- 记忆查询。
- 记忆更新。
- 记忆删除。
- 项目记忆。
- 每日总结。

4. Agent Runtime

负责：

- 任务规划。
- 工具调用。
- 执行结果验证。
- 任务状态持久化。
- 错误处理。

这四个模块应当相互协作，但避免相互依赖内部实现。

十一、还有一个很有意思的功能：AI 的「个人空间」

既然你想让它有生活感，可以为桌宠设计一个虚拟房间或工作空间。

它不一定需要 3D。

可以是简单的 2D 场景。

比如：

AI 工作时，在虚拟桌子旁敲键盘。

整理记忆时，显示阅读笔记的动作。

休息时，坐在椅子上。

用户进入工作区时，角色切换到助手模式。

甚至可以随着项目进展，在房间里展示获得的纪念物。

例如第一次完成软件版本后，房间里增加一个小奖杯。

这种设计能增加陪伴体验，但不必让 AI 假装自己真的拥有独立于软件之外的生活。

十二、你未来可以实现的一个完整场景

假设你已经使用软件三个月。

早上打开电脑。

紫发猫耳桌宠出现在桌面。

它说：

“早上好！昨天的模型动画系统已经测试完成了。不过还有一个问题需要你确认。”

你打开工作区。

AI 展示：

```text
昨晚工作报告

已完成：
✓ 分析头发物理参数
✓ 修改测试分支
✓ 运行基础测试
✓ 生成效果对比

等待确认：
○ 是否合并动画修改

今日计划：
1. 优化手部动画
2. 测试语音与表情同步
3. 整理项目文档
```

你说：

“先不用工作，陪我聊会儿。”

AI 暂停可中断的后台任务。

角色退出 Bongo Cat 模式，恢复普通待机。

你们聊了一会儿。

然后你说：

“好了，继续工作吧。”

AI 恢复之前的任务。

这就是我认为你的项目最理想的使用体验之一。

最后，我想给你一个更长远的建议

不要把 AI 的自主性设计成“它可以随便做任何事情”。

真正优秀的自主 AI 应该具备四种能力：

1. 知道自己正在做什么。
2. 知道哪些事情允许自己做。
3. 知道什么时候应该停止并询问用户。
4. 能够准确记录自己做过什么。

这四点，比让 AI 每时每刻都表现得像人更重要。

另外，你既然希望未来让它成为家人般的陪伴者，我建议你从一开始就支持**角色数据迁移**。

比如未来 DeepSeek 模型升级了，或者你更换了新的语言模型，角色的记忆、人物设定、共同经历和项目任务都不应该随之消失。

换句话说：

语言模型可以更换，但用户长期培养的 AI 角色应当能够延续。

这可能会成为你整个软件最有价值的设计之一。

你可以生成一个pdf总结我们刚刚的讨论吗?

gpt-5.5-instant

可以。我可以把刚才的讨论整理成一份完整的中文 PDF 方案文档，暂定标题为：

《AI 桌面陪伴智能体：完整产品规划与技术架构设计方案》

内容将涵盖：

1. **项目定位与最终愿景**：从 Live2D 桌宠发展为具有长期记忆、自主工作能力和家人般陪伴体验的 AI。
2. **Live2D 角色引擎**：自制 C 语言底层、动画混合、表情、口型同步、Bongo Cat 模式。
3. **AI 核心架构**：人格系统、认知上下文、模型路由、本地与云端模型。
4. **长期记忆系统**：用户记忆、共同经历、项目记忆、记忆整理与遗忘机制。
5. **自主生活与工作系统**：后台任务、事件驱动、主动交互、任务恢复、每日总结。
6. **Agent 与 Skills**：多智能体协作、MCP、工具调用和权限控制。
7. **语音与多模态**：实时语音通话、视觉感知、生图、生视频、生建模接口。

8. **软件界面设计**：桌宠模式、工作区、对话区、多 AI 群聊、思考区、记忆区。
9. **与 Neuro-sama 的对比**：现有优势、功能差距及未来发展方向。
10. **C 语言技术架构**：模块划分、数据结构、事件总线和后台服务。
11. **开发路线图**：从基础桌宠到高度自主的 AI 伙伴。
12. **AI 开发 Agent 指令**：可直接交给编程 Agent 使用的开发规范和任务说明。

我还会加入架构示意图、模块关系、开发优先级及具体实现建议，让它不仅是讨论总结，也能作为后续开发的项目设计文档。

不过，当前对话环境不能直接生成并附加可下载的 PDF 文件。

我可以先生成完整的中文文档，并提供一个可直接转换为 PDF 的 HTML 文件内容。你只需要用浏览器打开，按 `Ctrl + P`，选择“另存为 PDF”即可。

建议制作成 **20～30 页左右的正式项目规划书**，保留我们讨论过的重要技术细节，而不只是简单摘要。

